

Dynamic Vector User Guide

Code is in BETA State

**Non-backward compatible changes
may still be made**

User Guide for Release 2026.05

By

Jim Lewis

SynthWorks VHDL Training

Jim@SynthWorks.com

<http://www.SynthWorks.com>

Table of Contents

1	DynamicVectorGenericPkg Overview.....	4
2	Basic Usage.....	4
2.1	Package References	4
2.2	Declaring a VectorID	4
2.3	NewID: Construct the DynamicVector.....	4
2.4	Append: Adding Items to the End of the Vector.....	5
2.5	Get: Reading from the Vector	5
2.6	Set: Writing to the Vector	5
2.7	GetSize: Number of elements in the Vector	5
2.8	GetCapacity: Number of elements allocated for the Vector	5
2.9	Putting the basic pieces together	5
3	Append, Insert, Prepend, and Delete.....	7
3.1	Insert: Adding at a specified Index.....	7
3.2	Prepend: Insert values at the start of the vector	7
3.3	Delete: Remove values at a specified Index	7
4	Indexing, Index Operations, and Iterators.....	8
4.1	CopyID: Create new iterator	8
4.2	GetFirstIndex: Return first index of the vector.....	8
4.3	GetLastIndex: Last index of the vector	8
4.4	GetIndex: Iterator index value	8
4.5	SetIndex: Set iterator index value	9
4.6	HasNext.....	9
4.7	IndexNext.....	9
4.8	GetNext	9
4.9	HasPrevious	9
4.10	IndexPrevious	9
4.11	GetPrevious.....	9
4.12	Deallocate.....	10
5	Find & Match.....	10
5.1	Find.....	10
5.2	FindNext.....	10

5.3	FindPrevious	10
5.4	Match	10
6	Vector Information Operations	11
6.1	IsInitialized	11
6.2	IsEmpty	11
6.3	MakeEmpty	11
6.4	GetSize	11
6.5	GetCapacity.....	11
6.6	GetAlertLogID	11
7	Writing/Reading Vector Values	12
8	Generics and Instances of DynamicVectorGenericPkg.....	12
8.1	Generic Interface.....	12
8.2	Instances of DynamicVectorGenericPkg	12
9	Compiling and Using OSVVM Utility Library	12
10	About the Author - Jim Lewis.....	13

1 DynamicVectorGenericPkg Overview

A dynamic vector is a resizable vector. Like a vector it supports random access to elements via indices. The indices are numbered 0 to N-1, where N is the number of elements in the vector. As a data structure, it supports insertion and deletion, but it is optimized for insertion and deletion at the end of the vector (index N-1).

Dynamic vectors are implemented in a singleton data structure and can be passed around using their ID. This makes suitable for handling burst transfer information – particularly if the bursts are different sizes. As a singleton, different aspects of the verification environment such as packet generator, verification component (VC) driving the packet, and VC receiving and checking the packet can all reference the packet of data using its ID.

Dynamic vectors have similarities to C/Rust/Ada vectors, SystemVerilog dynamic arrays, Python lists, and Java VectorLists.

2 Basic Usage

2.1 Package References

Using dynamic vectors requires a package instance (provided for integer_vector and std_logic_vector) and a package reference. Note these are not included in the OsvvmContext.

```
Library osvvm ;
use osvvm.DynamicVectorPkg_IntV.all ;
architecture Test1 of tb is
```

2.2 Declaring a VectorID

A VectorID is a reference to the dynamic vector that is created internal to an instance of DynamicVectorGenericPkg (such as DynamicVectorPkg_IntV). The VectorID has the type DynamicVectorIDType. DynamicVectorIDType is an ordinary type and can be held in a signal or variable.

```
architecture Model of Sram is
    signal VectorID : VectorIDType ;
    . . .
```

2.3 NewID: Construct the Dynamic Vector

NewID allocates and initializes the dynamic vector data structure. It must be called before any dynamic vector operations are used.

```
VectorID <= NewID(
    Name      => "Packet1",
    Capacity  => 0,
    ParentID  => TbID) ;
```

2.4 **Append: Add to the End of the Vector**

Append adds either elements or vectors to the end of the Vector. Append increases the size of the vector.

```
Append(VectorID, ElementValue) ;  
Append(VectorID, VectorValue) ;
```

2.5 **Get: Read from a specified Index of the Vector**

The function Get reads either an element or a vector from an Index location in the Vector. The Index location must already exist.

```
ElementValue := Get(VectorID, Index) ;  
VectorValue := Get(VectorID, Index, VectorValue'length) ;
```

2.6 **Set: Write to a specified Index of the Vector**

The procedure Set writes either an element or a vector to an Index location in the vector. Set overwrites any previous values. The Index location must already exist.

```
Set(VectorID, Index, ElementValue) ;  
Set(VectorID, Index, VectorValue) ;
```

2.7 **GetSize: Number of elements in the Vector**

The function GetSize returns the number of values in the Vector. The vector has the indices 0 to GetSize -1.

```
Size := GetSize(VectorID) ;
```

2.8 **GetCapacity: Number of elements allocated for the Vector**

The capacity is the number of elements currently allocated for the vector. When the vector is constructed, the Capacity is initialized to 16. During an Append, Insert, or Prepend, if the vector size exceeds the capacity, the capacity is doubled. By doubling the capacity amortizes the costs of increasing the capacity and makes Append an $O(1)$ operation on average.

```
Capacity := GetCapacity (VectorID) ;
```

2.9 **Putting the basic pieces together**

The following declares the VectorID (variable VectorID), constructs it (NewID), adds 3 elements to the vector (Append), checks the first and last value (using Get), and changes the value at index 1 to 25 (Set).

```
library OSVVM ;  
context osvvm.OsvvmContext ;  
use osvvm.GenericVectorPkg_IntV.all ;  
  
entity Tb_Basic_1 is  
end Tb_Basic_1 ;
```

```

architecture Test1 of Tb_Basic_1 is
begin

    TestProc : process
        -- Declare the Vector ID
        variable VectorID : DynamicVectorIDType ;
        variable TbID      : AlertLogIDType ;
        variable Data      : integer ;
    begin

        SetTestName("Tb_Basic_1") ;
        TranscriptOpen ;
        SetLogEnable(PASSED, TRUE) ;
        SetTranscriptMirror(TRUE) ;
        TbID := NewID("TbID") ;

        -- Construct the Vector
        VectorID := NewID("ID", 0, TbID) ;

        -- Add 3 elements to the Vector
        Append(VectorID, 10) ;
        Append(VectorID, 20) ;
        Append(VectorID, 30) ;
        AffirmIfEqual(TbID, GetSize(VectorID), 3, "GetSize") ;

        -- Get the first and last elements of the vector
        AffirmIfEqual(TbID, Get(VectorID, 0), 10, "Get") ;
        AffirmIfEqual(TbID, Get(VectorID, 2), 30, "Get") ;

        -- Change the value at index 1 to 25
        Set(VectorID, 1, 25) ;

        TranscriptClose ;

        AffirmIfTranscriptsMatch(PATH_TO_VALIDATED_RESULTS) ;

        EndOfTestReports ;
        std.env.stop ;
        wait ;
    end process ControlProc ;

end Test1 ;

```

3 Append, Insert, Prepend, and Delete

Append, Insert, Prepend, and Delete increase or decrease the size of the vector. This can result in shifting values and memory allocation. Most of these operations are $O(n)$ – meaning take an amount of time that is proportional to the size of the vector.

For Append, which adds values to the end of the vector, this time can be amortized to $O(1)$ – meaning constant time access independent of the vector size – by doubling the capacity of the vector (amount of information that the vector can store) every time the size exceeds capacity. As a result, the allocate/deallocate and copy that can occur with append occurs infrequently.

For insert, prepend, and delete that occur at random locations in the vector or at the beginning, there will always be shifting of $O(n)$ values. Note that these operations can impact any index or iterator being used to access the vector.

3.1 Insert: Add value(s) at a specified Index

Insert adds either elements or vectors at the index specified (before any values that were previously at that index).

```
Insert(VectorID, Index, ElementValue) ;  
Insert(VectorID, Index, VectorValue) ;
```

3.2 Prepend: Insert value(s) at the start of the vector

Prepend adds either elements or vectors at the start of the vector (before any values that were previously at the start).

```
Prepend(VectorID, ElementValue) ;  
Prepend(VectorID, VectorValue) ;
```

3.3 Delete: Remove values at a specified Index

Delete removes either elements or vectors at the index specified.

```
Delete(VectorID, Index) ;           -- Single Value  
Delete(VectorID, Index, NumValues) ; -- Vector of Values
```

4 Indexing, Index Operations, and Iterators

Get and set can access vector values at an index location. Assuming the vector currently contains 8 8-bit values, then a for loop can be used to iterate across all values in the vector.

```
for i in 0 to 7 loop
    DataSlv8 := Get(VectorID1, i*8, 8) ; --
    Send(StreamTx, DataSlv8) ;
end loop ;
```

Each ID of the data structure has an iterator associated with it. The iterator supports sequencing across the data structure getting one item at a time. As values are set/get the iterator updates. The following accomplishes the sequencing as the previous task.

```
SetIndex(VectorID1) ; -- restore iterator to 0 index
while HasNext(VectorID1) loop
    DataSlv8 := GetNext(VectorID1, 8) ;
    Send(StreamTx, DataSlv8) ;
end loop ;
```

The initial iterator is created by NewID when the vector is constructed. If multiple iterators are desirable, then additional iterators can be created by CopyID. The ID returned by CopyID references the same vector as the original ID - hence, the copy is referred to as a shallow copy.

The vector exists as long as there is an ID that references it. Deallocate releases an ID. When Deallocate is called on the last iterator associated with an ID, the vector is deallocated.

4.1 CopyID: Create new iterator

CopyID creates a new iterator for the dynamic vector data structure.

```
CopyID1 := CopyID( VectorID1) ;
```

4.2 GetFirstIndex: Return first index of the vector

Returns the first index of the vector. This is 0 in the current implementation.

```
Index := GetFirstIndex( VectorID1) ;
```

4.3 GetLastIndex: Last index of the vector

Set NumValues parameter to the number of elements in the value (1 for an element), and GetLastIndex will return the location of the last index.

```
Index := GetLastIndex( VectorID1, NumValues => 1) ;
```

4.4 GetIndex: Iterator index value

Returns the current value of the Iterator index.

```
Index := GetIndex( VectorID1) ;
```

4.5 **SetIndex: Set iterator index value**

Set the iterator index value. Without a parameter, it sets the value to 0.

```
SetIndex( VectorID1) ;  
SetIndex( VectorID1, GetLastIndex(VectorID1)) ;
```

4.6 **HasNext**

Return TRUE when if there are NumValues (default = 1) available at the current iterator.

```
while HasNext(VectorID1, NumValues) loop  
    . . .
```

4.7 **IndexNext**

Return the current iterator value and then increment the iterator by NumValues (default = 1).

```
Index := IndexNext(VectorID1, NumValues) ;
```

4.8 **GetNext**

Return NumValues elements using the current iterator and then increment the iterator by NumValues (default = 1).

```
DataElement := GetNext(VectorID1) ;  
DataVector  := GetNext(VectorID1, NumValues) ;
```

4.9 **HasPrevious**

Return TRUE when if there are NumValues (default = 1) available before the current iterator.

```
while HasPrevious(VectorID1, NumValues) loop  
    . . .
```

4.10 **IndexPrevious**

Decrement the current iterator by NumValues (default = 1) and return the iterator value.

```
Index := IndexPrevious(VectorID1, NumValues) ;
```

4.11 **GetPrevious**

Decrement the iterator by NumValues (default = 1) and then return NumValues elements using the current iterator.

```
DataElement := GetPrevious(VectorID1) ;  
DataVector  := GetPrevious(VectorID1, NumValues) ;
```

4.12 Deallocate

Deallocate releases an ID. When Deallocate is called on the last iterator associated with an ID, the vector is deallocated. The vector exists as long as there is an ID that references it.

```
VectorID1 := Deallocate(VectorID1) ;  
CopyID1   := Deallocate(CopyID1) ;
```

5 Find & Match

Find searches for a value and returns the index where it was found or -1 if not found. Match gets a value from an index and if it matches the value it returns true.

5.1 Find

Find searches from GetFirstIndex to GetLastIndex for a value and returns the index where it was found or -1 if not found.

```
Index := Find( VectorID1, ElementValue) ;  
Index := Find( VectorID1, VectorValue) ;
```

5.2 FindNext

FindNext searches from the current iterator value to GetLastIndex for a value and returns the index where it was found or -1 if not. The iterator is updated using IndexNext. Allows finding the same value multiple times within a vector.

```
Index := FindNext( VectorID1, ElementValue) ;  
Index := FindNext( VectorID1, VectorValue) ;
```

5.3 FindPrevious

FindPrevious searches from the current iterator value to GetFirstIndex for a value and returns the index where it was found or -1 if not. The iterator is updated using IndexPrevious. Allows finding a value multiple times within a vector.

```
Index := FindPrevious( VectorID1, ElementValue) ;  
Index := FindPrevious ( VectorID1, VectorValue) ;
```

5.4 Match

Match gets a value from an index and if it matches the value it returns true.

```
if Match( VectorID1, Index, ElementValue) then  
if Match( VectorID1, Index, VectorValue) then
```

6 Information About the Vector

6.1 **IsInitialized**

Returns TRUE when the ID has been initialized.

```
if IsInitialized(VectorID1) then  
if IsInitialized(CopyID1) then
```

6.2 **IsEmpty**

Returns TRUE when there are no elements in the vector.

```
if IsEmpty(VectorID1) then  
if IsEmpty(CopyID1) then
```

6.3 **MakeEmpty**

Remove all elements from the vector. Impacts all iterators to the vector.

```
MakeEmpty(VectorID1) ;
```

6.4 **GetSize**

Returns the number of values in the vector.

```
Size := GetSize(VectorID1) ;
```

6.5 **GetCapacity**

Returns the capacity in the vector. During Append, Insert, Prepend, if size is greater than capacity, the capacity is doubled. This results in allocating a vector that matches the new capacity, copying the old values to it, and deallocating the old vector.

```
Capacity := GetCapacity(VectorID1) ;
```

6.6 **GetAlertLogID**

Returns the AlertLogID of the vector data structure that is either created or copied during the call to NewID.

```
VectorAlertLogID := GetAlertLogID(VectorID1) ;
```

7 Writing/Reading Vector Values

Note this feature reflects the current implementation and may change.

Vector values are written and read so that the right most value is stored first. When the vector elements are integer, this is not so important. When the vector elements are `std_logic`, if the low bytes are written into the vector first and each element is written low bit first, then when 32-bit words are read out (and the bits are reversed), then the 32-bit word is in the proper order – without any transformations required.

8 Generics and Instances of DynamicVectorGenericPkg

8.1 Generic Interface

The generic interface for `DynamicVectorGenericPkg` is shown below.

```
package DynamicVectorGenericPkg is
    generic (type VectorType is vector (type is range <>) of type is private ) ;
    . . .
```

8.2 Instances of DynamicVectorGenericPkg

There are two primary instances `DynamicVectorPkg_IntV` and `DynamicVectorPkg_slv`. These are shown below.

```
package DynamicVectorPkg_IntV is new work.DynamicVectorGenericPkg
    generic map (VectorType => integer_vector ) ;

library ieee ;
use ieee.std_logic_1164.all ;
package DynamicVectorPkg_slv is new work.DynamicVectorGenericPkg
    generic map (VectorType => std_logic_vector ) ;
```

9 Compiling and Using OSVVM Utility Library

Reference all packages in the OSVVM Utility library by using the context declaration:

```
library OSVVM ;
context osvvm.OSvvmContext ;
```

Compilation order for OSVVM Utility Library is in `OSVVM_release_notes.pdf`. Rather than learning this, we recommend using the OSVVM compilation scripts.

OSVVM Utility library is released under the Apache open source license. It is free (both to download and use - there are no license fees). You can download it from osvvm.org or from our development area on GitHub (<https://github.com/OSVVM/OSVVM>).

If you add features to the package, please donate them back under the same license as candidates to be added to the standard version of the package. If you need features, be sure to contact us.

We also support the OSVVM user community and blogs through <http://www.osvvm.org>. Interested in sharing about your experience using OSVVM? Let us know, you can blog about it at [osvvm.org](http://www.osvvm.org).

10 About the Author - Jim Lewis

Jim Lewis is an innovator and leader in the VHDL community. He has 40 plus years of design and teaching experience and is well known within the VHDL community. He is the chief architect of the Open Source VHDL Verification Methodology (OSVVM). He has been an active member of VHDL's standards for over 25 years. He is an expert VHDL trainer for SynthWorks Design Inc. In his design practice, he has created designs for print servers, networking, fighter jets, video phones, and spacecraft.

Whether teaching, developing OSVVM, doing consulting work, or working on the IEEE VHDL standard, Jim brings a deep understanding of all things VHDL, and is expert at architecting solutions that solve difficult problems in simple ways.

Please support our volunteer efforts by buying your VHDL training from SynthWorks.

If you find bugs these packages or would like to request enhancements, you can reach me at jim@synthworks.com.